

布尔莎七参数坐标转换报告

一、运行方式

运行环境：VS2022 C#

可运行程序文件：布尔莎七参数坐标转换代码\bin\Debug\ Coordinate Transformation.exe

执行方式：

1. 输入 旧点.txt;
2. 输入 新点.txt;
3. 计算七参数;
4. 转换未知坐标。

二、原理

布尔莎七参数坐标转换是一种用于不同坐标系之间转换的常用方法，适用于空间直角坐标系的转换。它基于七参数模型，这七个参数分别是三个平移参数（ ΔX 、 ΔY 、 ΔZ ）、三个旋转参数（ ϵ_X 、 ϵ_Y 、 ϵ_Z ）和一个尺度参数（ m ）。

其数学模型可以表示为：

$$\begin{bmatrix} X_2 \\ Y_2 \\ Z_2 \end{bmatrix} = \begin{bmatrix} \Delta X \\ \Delta Y \\ \Delta Z \end{bmatrix} + (1 + m) \begin{bmatrix} 1 & \epsilon_Z & -\epsilon_Y \\ -\epsilon_Z & 1 & \epsilon_X \\ \epsilon_Y & -\epsilon_X & 1 \end{bmatrix} \begin{bmatrix} X_1 \\ Y_1 \\ Z_1 \end{bmatrix}$$

其中， (X_1, Y_1, Z_1) 是原坐标系中的坐标， (X_2, Y_2, Z_2) 是目标

坐标系中的坐标。

在实际计算中，通常需要至少三个公共点（即同时具有原坐标系和目标坐标系坐标的点）来求解这七个参数。通过构建误差方程，利用最小二乘法求解法方程，从而得到七参数的值。

三、代码

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Windows.Forms;
namespace CoordinateTransformation
{
    public partial class Form1 : Form
    {
        private List<PointData> oldPoints = new List<PointData>();
        private List<PointData> newPoints = new List<PointData>();
        private List<PointData> transformPoints = new List<PointData>();
        private TransformationParameters parameters = new TransformationParameters();
        private PointData oldCentroid;
        private PointData newCentroid;
        public Form1()
        {
            InitializeComponent();
        }
        #region 事件处理
        private void btnLoadOldPoints_Click(object sender, EventArgs e)
        {
            if (openFileDialog1.ShowDialog() == DialogResult.OK)
            {
                try
                {
                    oldPoints = ReadPointData(openFileDialog1.FileName);
                    oldPoints = RemoveOutliers(oldPoints);
                    txtOldPointsPath.Text = openFileDialog1.FileName;
                    DisplayPoints(lvOldPoints, oldPoints);
                    MessageBox.Show("旧点数据加载成功", "成功", MessageBoxButtons.OK, MessageBoxIcon.Information);
                }
            }
        }
    }
}
```

```

        catch (Exception ex)
        {
            MessageBox.Show($"加载旧点数据失败: {ex.Message}", "错误", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}

private void btnLoadNewPoints_Click(object sender, EventArgs e)
{
    if (openFileDialog1.ShowDialog() == DialogResult.OK)
    {
        try
        {
            newPoints = ReadPointData(openFileDialog1.FileName);
            newPoints = RemoveOutliers(newPoints);
            txtNewPointsPath.Text = openFileDialog1.FileName;
            DisplayPoints(lvNewPoints, newPoints);
            MessageBox.Show("新点数据加载成功", "成功", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        catch (Exception ex)
        {
            MessageBox.Show($"加载新点数据失败: {ex.Message}", "错误", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}

private void btnCalculate_Click(object sender, EventArgs e)
{
    if (oldPoints.Count == 0 || newPoints.Count == 0)
    {
        MessageBox.Show("请先加载旧点和新点数据", "提示", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }
    if (oldPoints.Count != newPoints.Count)
    {
        MessageBox.Show("旧点和新点数量不一致", "错误", MessageBoxButtons.OK, MessageBoxIcon.Error);
        return;
    }
    if (oldPoints.Count < 3)
    {

```

```

        MessageBox.Show("至少需要 3 个公共点来计算七参数", "错误", MessageBoxButtons.OK, Message
eBoxIcon.Error);
        return;
    }
    try
    {
        CalculateTransformationParameters();
        DisplayParameters();
        CalculateAndDisplayErrors();
        MessageBox.Show("七参数计算完成", "成功", MessageBoxButtons.OK, MessageBoxIcon.Informati
on);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"计算七参数失败: {ex.Message}", "错误", MessageBoxButtons.OK, MessageB
oxIcon.Error);
    }
}

private void btnTransform_Click(object sender, EventArgs e)
{
    if (parameters.Scale == 0)
    {
        MessageBox.Show("请先计算七参数", "提示", MessageBoxButtons.OK, MessageBoxIcon.Warnin
g);
        return;
    }
    try
    {
        ParseTransformPoints();
        if (transformPoints.Count == 0) return;
        TransformPoints();
        DisplayTransformedPoints();
        MessageBox.Show($"成功转换 {transformPoints.Count} 个点", "成功", MessageBoxButtons.OK, M
essageBoxIcon.Information);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"坐标转换失败: {ex.Message}", "错误", MessageBoxButtons.OK, MessageB
oxIcon.Error);
    }
}

#endregion
#region 数据处理
private List<PointData> ReadPointData(string filePath)

```

```

{
    return File.ReadAllLines(filePath)
        .Where(line => !string.IsNullOrEmpty(line))
        .Select(line => line.Split(new[] { ' ', '\t' }, StringSplitOptions.RemoveEmptyEntries))
        .Where(parts => parts.Length >= 4)
        .Select(parts => new PointData
        {
            ID = parts[0],
            X = double.Parse(parts[1]),
            Y = double.Parse(parts[2]),
            Z = double.Parse(parts[3])
        }).ToList();
}

private void DisplayPoints(ListView listView, List<PointData> points)
{
    listView.Items.Clear();
    foreach (var point in points)
    {
        var item = new ListViewItem(point.ID);
        item.SubItems.AddRange(new[] {
            point.X.ToString("F6"),
            point.Y.ToString("F6"),
            point.Z.ToString("F6")
        });
        listView.Items.Add(item);
    }
}

private void ParseTransformPoints()
{
    transformPoints.Clear();
    var lines = txtTransformPoints.Text.Split(new[] { Environment.NewLine }, StringSplitOptions.RemoveEmptyEntries);
    foreach (var line in lines)
    {
        if (string.IsNullOrEmpty(line) || line.StartsWith("示例")) continue;
        var parts = line.Split(new[] { ' ', '\t' }, StringSplitOptions.RemoveEmptyEntries);
        if (parts.Length >= 4)
        {
            transformPoints.Add(new PointData
            {
                ID = parts[0],
                X = double.Parse(parts[1]),
                Y = double.Parse(parts[2]),
                Z = double.Parse(parts[3])
            });
        }
    }
}

```

```

        });
    }
}

if (transformPoints.Count == 0)
    throw new InvalidOperationException("未找到有效的待转换点数据");
}

private List<PointData> RemoveOutliers(List<PointData> points)
{
    double meanX = points.Average(p => p.X);
    double meanY = points.Average(p => p.Y);
    double meanZ = points.Average(p => p.Z);
    double stdDevX = Math.Sqrt(points.Average(p => Math.Pow(p.X - meanX, 2)));
    double stdDevY = Math.Sqrt(points.Average(p => Math.Pow(p.Y - meanY, 2)));
    double stdDevZ = Math.Sqrt(points.Average(p => Math.Pow(p.Z - meanZ, 2)));
    return points.Where(p =>
        Math.Abs(p.X - meanX) <= 3 * stdDevX &&
        Math.Abs(p.Y - meanY) <= 3 * stdDevY &&
        Math.Abs(p.Z - meanZ) <= 3 * stdDevZ
    ).ToList();
}

#endregion
#region 七参数计算核心逻辑
private void CalculateTransformationParameters()
{
    int maxIterations = 10;
    double tolerance = 1e-8;
    for (int iter = 0; iter < maxIterations; iter++)
    {
        oldCentroid = CalculateCentroid(oldPoints);
        newCentroid = CalculateCentroid(newPoints);
        var oldCentered = CenterPoints(oldPoints, oldCentroid);
        var newCentered = CenterPoints(newPoints, newCentroid);
        var (A, L) = BuildCoefficientMatrix(oldCentered, newCentered);
        var X = SolveLinearEquation(A, L);
        // 提取七参数
        parameters.Dx = X[0];
        parameters.Dy = X[1];
        parameters.Dz = X[2];
        parameters.Rx = X[3];
        parameters.Ry = X[4];
        parameters.Rz = X[5];
        parameters.Scale = 1.0 + X[6] / 1000000.0;
        // 计算残差
        double sumResidual = 0;
    }
}

```

```

        for (int i = 0; i < oldPoints.Count; i++)
        {
            PointData oldPoint = oldPoints[i];
            PointData newPoint = newPoints[i];
            PointData transformedPoint = TransformPoint(oldPoint);
            double residualX = newPoint.X - transformedPoint.X;
            double residualY = newPoint.Y - transformedPoint.Y;
            double residualZ = newPoint.Z - transformedPoint.Z;
            sumResidual += residualX * residualX + residualY * residualY + residualZ * residualZ;
        }
        // 判断是否收敛
        if (sumResidual < tolerance)
        {
            break;
        }
    }
}

private PointData CalculateCentroid(List<PointData> points)
{
    return new PointData
    {
        X = points.Average(p => p.X),
        Y = points.Average(p => p.Y),
        Z = points.Average(p => p.Z)
    };
}

private List<PointData> CenterPoints(List<PointData> points, PointData centroid)
{
    return points.Select(p => new PointData
    {
        ID = p.ID,
        X = p.X - centroid.X,
        Y = p.Y - centroid.Y,
        Z = p.Z - centroid.Z
    }).ToList();
}

private (double[,], double[]) BuildCoefficientMatrix(List<PointData> oldPoints, List<PointData> newPoints)
{
    int n = oldPoints.Count;
    double[,] A = new double[3 * n, 7];
    double[] L = new double[3 * n];
    for (int i = 0; i < n; i++)
    {

```

```

        double x = oldPoints[i].X;
        double y = oldPoints[i].Y;
        double z = oldPoints[i].Z;
        // X 方程系数
        A[i * 3, 0] = 1; A[i * 3, 3] = 0; A[i * 3, 4] = z; A[i * 3, 5] = -y; A[i * 3, 6] = x;
        // Y 方程系数
        A[i * 3 + 1, 1] = 1; A[i * 3 + 1, 3] = -z; A[i * 3 + 1, 4] = 0; A[i * 3 + 1, 5] = x; A
[i * 3 + 1, 6] = y;
        // Z 方程系数
        A[i * 3 + 2, 2] = 1; A[i * 3 + 2, 3] = y; A[i * 3 + 2, 4] = -x; A[i * 3 + 2, 5] = 0; A
[i * 3 + 2, 6] = z;
        L[i * 3] = newPoints[i].X;
        L[i * 3 + 1] = newPoints[i].Y;
        L[i * 3 + 2] = newPoints[i].Z;
    }
    return (A, L);
}
private double[] SolveLinearEquation(double[,] A, double[] L)
{
    int rows = A.GetLength(0);
    int cols = A.GetLength(1);
    // 构建法方程  $NX = B$ 
    double[,] N = new double[cols, cols];
    double[] B = new double[cols];
    for (int i = 0; i < cols; i++)
    {
        for (int j = 0; j <= i; j++)
        {
            double sum = 0;
            for (int k = 0; k < rows; k++)
                sum += A[k, i] * A[k, j];
            N[i, j] = sum;
            N[j, i] = sum; // 对称矩阵
        }
        double sumB = 0;
        for (int k = 0; k < rows; k++)
            sumB += A[k, i] * L[k];
        B[i] = sumB;
    }
    // 求解线性方程组
    return SolveNormalEquation(N, B);
}
private double[] SolveNormalEquation(double[,] N, double[] B)
{

```

```
int n = N.GetLength(0);
double[] X = new double[n];
// 高斯消元法求解
for (int i = 0; i < n; i++)
{
    // 选主元
    int maxRow = i;
    for (int k = i + 1; k < n; k++)
    {
        if (Math.Abs(N[k, i]) > Math.Abs(N[maxRow, i]))
        {
            maxRow = k;
        }
    }
    // 交换行
    if (maxRow != i)
    {
        for (int j = 0; j < n; j++)
        {
            double temp = N[i, j];
            N[i, j] = N[maxRow, j];
            N[maxRow, j] = temp;
        }
        double tempB = B[i];
        B[i] = B[maxRow];
        B[maxRow] = tempB;
    }
    // 消元
    for (int k = i + 1; k < n; k++)
    {
        double factor = N[k, i] / N[i, i];
        for (int j = i; j < n; j++)
        {
            N[k, j] -= factor * N[i, j];
        }
        B[k] -= factor * B[i];
    }
}
// 回代求解
for (int i = n - 1; i >= 0; i--)
{
    double sum = 0;
    for (int j = i + 1; j < n; j++)
    {

```

```

        sum += N[i, j] * X[j];
    }
    X[i] = (B[i] - sum) / N[i, i];
}
return X;
}
private PointData TransformPoint(PointData point)
{
    double dx = point.X - oldCentroid.X;
    double dy = point.Y - oldCentroid.Y;
    double dz = point.Z - oldCentroid.Z;
    double rx = parameters.Rx * Math.PI / 180.0;
    double ry = parameters.Ry * Math.PI / 180.0;
    double rz = parameters.Rz * Math.PI / 180.0;
    double cosRx = Math.Cos(rx);
    double sinRx = Math.Sin(rx);
    double cosRy = Math.Cos(ry);
    double sinRy = Math.Sin(ry);
    double cosRz = Math.Cos(rz);
    double sinRz = Math.Sin(rz);
    double[] rotationMatrix = new double[9];
    rotationMatrix[0] = cosRy * cosRz;
    rotationMatrix[1] = cosRy * sinRz;
    rotationMatrix[2] = -sinRy;
    rotationMatrix[3] = sinRx * sinRy * cosRz - cosRx * sinRz;
    rotationMatrix[4] = sinRx * sinRy * sinRz + cosRx * cosRz;
    rotationMatrix[5] = sinRx * cosRy;
    rotationMatrix[6] = cosRx * sinRy * cosRz + sinRx * sinRz;
    rotationMatrix[7] = cosRx * sinRy * sinRz - sinRx * cosRz;
    rotationMatrix[8] = cosRx * cosRy;
    double[] transformed = new double[3];
    transformed[0] = parameters.Scale * (rotationMatrix[0] * dx + rotationMatrix[1] * dy + rotationMatrix[2] * dz) + parameters.Dx + newCentroid.X;
    transformed[1] = parameters.Scale * (rotationMatrix[3] * dx + rotationMatrix[4] * dy + rotationMatrix[5] * dz) + parameters.Dy + newCentroid.Y;
    transformed[2] = parameters.Scale * (rotationMatrix[6] * dx + rotationMatrix[7] * dy + rotationMatrix[8] * dz) + parameters.Dz + newCentroid.Z;
    return new PointData
    {
        ID = point.ID,
        X = transformed[0],
        Y = transformed[1],
        Z = transformed[2]
    };
};

```

```

    }
    private void DisplayParameters()
    {
        lvParameters.Items.Clear();
        lvParameters.Items.Add(new ListViewItem(new[] { "DX", (parameters.Dx*1000000).ToString("F6") }));
        lvParameters.Items.Add(new ListViewItem(new[] { "DY", (parameters.Dy * 1000000).ToString("F6") }));
        lvParameters.Items.Add(new ListViewItem(new[] { "DZ", (parameters.Dz*1000000).ToString("F6") }));
        lvParameters.Items.Add(new ListViewItem(new[] { "RX", (parameters.Rx*1000000).ToString("F6") }));
        lvParameters.Items.Add(new ListViewItem(new[] { "RY", (parameters.Ry * 1000000).ToString("F6") }));
        lvParameters.Items.Add(new ListViewItem(new[] { "RZ", (parameters.Rz * 1000000).ToString("F6") }));
        lvParameters.Items.Add(new ListViewItem(new[] { "Scale", (parameters.Scale * 1000000).ToString("F6") }));
    }
    private void CalculateAndDisplayErrors()
    {
        lvError.Items.Clear();
        double sumErrorX = 0;
        double sumErrorY = 0;
        double sumErrorZ = 0;
        for (int i = 0; i < oldPoints.Count; i++)
        {
            PointData oldPoint = oldPoints[i];
            PointData newPoint = newPoints[i];
            PointData transformedPoint = TransformPoint(oldPoint);
            double errorX = newPoint.X - transformedPoint.X;
            double errorY = newPoint.Y - transformedPoint.Y;
            double errorZ = newPoint.Z - transformedPoint.Z;
            sumErrorX += errorX * errorX;
            sumErrorY += errorY * errorY;
            sumErrorZ += errorZ * errorZ;
        }
        double mX = Math.Sqrt(sumErrorX / oldPoints.Count);
        double mY = Math.Sqrt(sumErrorY / oldPoints.Count);
        double mZ = Math.Sqrt(sumErrorZ / oldPoints.Count);
        lvError.Items.Add(new ListViewItem(new[] { "MX", mX.ToString("F6") }));
        lvError.Items.Add(new ListViewItem(new[] { "MY", mY.ToString("F6") }));
        lvError.Items.Add(new ListViewItem(new[] { "MZ", mZ.ToString("F6") }));
    }

```

```

private void TransformPoints()
{
    for (int i = 0; i < transformPoints.Count; i++)
    {
        transformPoints[i] = TransformPoint(transformPoints[i]);
    }
}

private void DisplayTransformedPoints()
{
    lvResult.Items.Clear();
    foreach (var point in transformPoints)
    {
        var item = new ListViewItem(point.ID);
        item.SubItems.AddRange(new[] {
            point.X.ToString("F6"),
            point.Y.ToString("F6"),
            point.Z.ToString("F6")
        });
        lvResult.Items.Add(item);
    }
}

#endregion
}

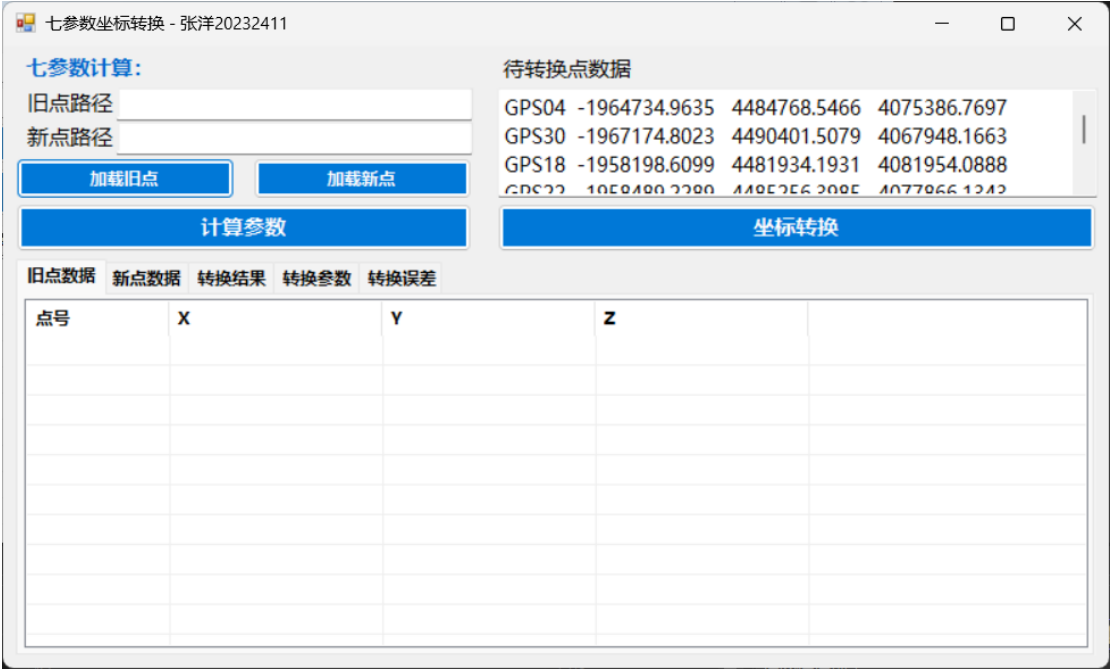
public class PointData
{
    public string ID { get; set; }
    public double X { get; set; }
    public double Y { get; set; }
    public double Z { get; set; }
}

public class TransformationParameters
{
    public double Dx { get; set; }
    public double Dy { get; set; }
    public double Dz { get; set; }
    public double Rx { get; set; }
    public double Ry { get; set; }
    public double Rz { get; set; }
    public double Scale { get; set; }
}
}

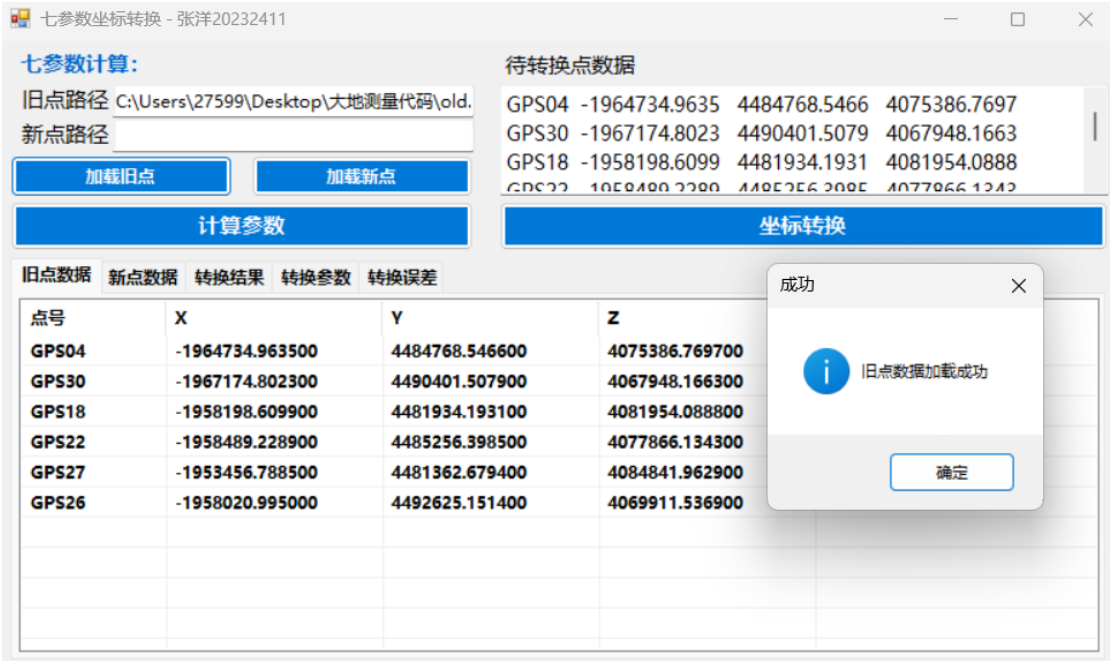
```

四、结果

初始界面：



加载旧点：



加载新点：

七参数坐标转换 - 张洋20232411

七参数计算:

旧点路径 C:\Users\27599\Desktop\大地测量代码\old.

新点路径 C:\Users\27599\Desktop\大地测量代码\new

加载旧点

加载新点

待转换点数据

GPS04	-1964734.9635	4484768.5466	4075386.7697
GPS30	-1967174.8023	4490401.5079	4067948.1663
GPS18	-1958198.6099	4481934.1931	4081954.0888
GPS22	-1958490.2200	4485256.2005	4077966.1242

计算参数

坐标转换

旧点数据

新点数据

转换结果

转换参数

转换误差

点号	X	Y	Z
GPS04	-1964642.835900	4484908.586000	4075486.898100
GPS30	-1967082.716000	4490541.646000	4068048.150900
GPS18	-1958106.370100	4482074.178900	4082054.321100
GPS22	-1958396.994900	4485396.445000	4077966.297100
GPS27	-1953364.459100	4481502.655000	4084942.264900
GPS26	-1957928.755100	4492765.305000	4070011.563000

成功

新点数据加载成功

确定

计算参数和误差：

七参数坐标转换 - 张洋20232411

七参数计算:

旧点路径 C:\Users\27599\Desktop\大地测量代码\old.

新点路径 C:\Users\27599\Desktop\大地测量代码\new

加载旧点

加载新点

待转换点数据

GPS04	-1964734.9635	4484768.5466	4075386.7697
GPS30	-1967174.8023	4490401.5079	4067948.1663
GPS18	-1958198.6099	4481934.1931	4081954.0888
GPS22	-1958490.2200	4485256.2005	4077966.1242

计算参数

坐标转换

旧点数据

新点数据

转换结果

转换参数

转换误差

参数	值
DX	-0.000310
DY	-0.000776
DZ	0.000155
RX	-0.925278
RY	0.323529
RZ	-0.887332
Scale	1000001.000018

七参数坐标转换 - 张洋20232411

七参数计算:

旧点路径 C:\Users\27599\Desktop\大地测量代码\old.
新点路径 C:\Users\27599\Desktop\大地测量代码\new

加载旧点

加载新点

计算参数

待转换点数据

GPS04 -1964734.9635 4484768.5466 4075386.7697
GPS30 -1967174.8023 4490401.5079 4067948.1663
GPS18 -1958198.6099 4481934.1931 4081954.0888
GPS22 -1958397.0178 4485396.4541 4077966.2752
GPS27 -1953364.5724 4481502.7311 4084942.1107
GPS26 -1957928.7835 4492765.2146 4070011.6699

旧点数据

新点数据

转换结果

转换参数

转换误差

指标	值
MX	0.075730
MY	0.064155
MZ	0.103968

新坐标转换:

七参数坐标转换 - 张洋20232411

七参数计算:

旧点路径 C:\Users\27599\Desktop\大地测量代码\old.
新点路径 C:\Users\27599\Desktop\大地测量代码\new

加载旧点

加载新点

计算参数

待转换点数据

GPS04 -1964734.9635 4484768.5466 4075386.7697
GPS30 -1967174.8023 4490401.5079 4067948.1663
GPS18 -1958198.6099 4481934.1931 4081954.0888
GPS22 -1958397.0178 4485396.4541 4077966.2752
GPS27 -1953364.5724 4481502.7311 4084942.1107
GPS26 -1957928.7835 4492765.2146 4070011.6699

旧点数据

新点数据

转换结果

转换参数

转换误差

点号	X	Y	Z
GPS04	-1964642.758697	4484908.601752	4075486.908088
GPS30	-1967082.599982	4490541.568768	4068048.297326
GPS18	-1958106.398554	4482074.245413	4082054.233746
GPS22	-1958397.017873	4485396.454197	4077966.275210
GPS27	-1953364.572420	4481502.731168	4084942.110752
GPS26	-1957928.783574	4492765.214601	4070011.669977

成功

成功转换 6 个点

确定